

PROJECT BASED LEARNING REPORT

Banking System with Fraud Detection using Machine Learning

Submitted by: Azim Aftab (1/24/SET/BCS/448),

Yash Gour (1/24/SET/BCS/450)

Subject: Python | Faculty: Swati Hans | Academic Year: 2025–2026

ABSTRACT

This project presents a detailed implementation of a smart banking system using Python integrated with Machine Learning techniques. The system performs essential banking operations such as account creation, secure login, deposit, withdrawal, and transaction tracking. A Random Forest Classifier is trained on a dataset of 50,000 synthetic transaction records to detect fraudulent activities. The model analyzes transaction patterns such as amount, transaction type, and time gap to predict fraud. Security is enhanced using SHA-256 hashing for PIN protection and an account lock mechanism after three failed attempts.

INTRODUCTION

With the rapid growth of digital transactions, ensuring security in banking systems has become crucial. Traditional systems rely on manual verification and lack intelligent fraud detection mechanisms. This project addresses these issues by integrating machine learning to automatically detect suspicious transactions in real time. It demonstrates how Python, data structures, and machine learning can be combined to build a secure and efficient banking system.

OBJECTIVES

- To develop a functional and secure banking system using Python.
- To implement Machine Learning techniques for fraud detection.
- To ensure secure authentication using hashing algorithms.
- To demonstrate Object-Oriented Programming concepts.
- To simulate real-world banking operations.

SYSTEM ANALYSIS

Traditional banking systems often lack real-time fraud detection and rely on delayed verification processes. This can lead to financial losses and security breaches. The proposed system uses predictive analysis to identify fraud based on transaction patterns such as unusually high amounts or suspicious withdrawals. This improves efficiency, reduces manual effort, and enhances system security.

METHODOLOGY

A dataset of 50,000 transactions is generated using NumPy. Each transaction includes attributes such as amount, transaction type, and time gap. Fraud labels are assigned based on predefined conditions. The dataset is split into training and testing sets using `train_test_split` from Scikit-learn. A Random Forest Classifier is trained to classify transactions as fraudulent or safe. During system operation, each transaction is evaluated by the model before execution.

IMPLEMENTATION

The system is implemented using Python programming language. Pandas is used for data handling, and NumPy is used for generating synthetic datasets. Scikit-learn is used to train the Random Forest model. The BankAccount class encapsulates all functionalities such as deposit, withdrawal, fraud detection, and transaction history management. A dictionary is used to store user accounts, and lists are used to maintain transaction records.

DATA STRUCTURES USED

- Dictionary: Used to store user accounts efficiently.
- List: Used for storing transaction history.
- DataFrame: Used for storing and processing the dataset for machine learning.

SECURITY FEATURES

- SHA-256 hashing ensures secure storage of user PINs.
- Account locking after three failed login attempts prevents unauthorized access.
- Fraud detection model blocks suspicious transactions before execution.

ER DIAGRAM

User → Account → Transaction → Fraud Detection Model → Result

FLOWCHART

Start → Create/Login → Verify PIN → Perform Transaction → Fraud Check → Allow/Block Transaction → End

OUTPUT

BANKING SYSTEM WITH FRAUD DETECTION

1.Create 2.Login 3.Exit

Choice: 1

Enter username: azim

Enter PIN: 1234

Account created successfully!

Choice: 2

Enter username: azim

Enter PIN: 1234

Welcome azim

Option: 1 (Deposit)

Amount: 5000

₹5000 deposited | Balance: ₹5000

Option: 2 (Withdraw)

Amount: 80000

Enter PIN: 1234

⚠️ FRAUD ALERT: Withdraw blocked!

RESULTS AND DISCUSSION

The system successfully performs banking operations and detects fraudulent transactions accurately. High-value and suspicious withdrawals are blocked, ensuring account safety. The integration of Machine Learning improves decision-making and enhances security.

CONCLUSION

This project demonstrates the practical application of Python, Machine Learning, and Data Structures in building a secure banking system. It highlights how intelligent systems can prevent fraud and improve efficiency.

FUTURE SCOPE

- Integration with real databases.
- Development of graphical user interface.
- Use of advanced machine learning models.
- Deployment as a web or mobile application.

REFERENCES

1. Data Structures and Algorithms – Textbooks
2. C programming basics – GeeksforGeeks, Tutorialspoint
3. Articles on game logic using arrays
4. Research references on Minimax algorithm
5. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts.
McGraw-Hill Education.
6. MySQL Documentation: <https://dev.mysql.com/doc/>
7. W3Schools SQL Tutorial: <https://www.w3schools.com/sql/>
8. GeeksforGeeks — Data Structures & Algorithms
- 9.